Chapter 17

A USER INTERFACE

The functionality described in the preceding chapters is useful for a user who can develop his/her own applications software on top of them, but not for the ordinary end user who does not wish to write his/her own programs. To satisfy the needs of such a user, we must develop yet a higher layer of GWB, its *user interface*.

17.1 LEVELS OF A MODELER REVISITED

One of the underlying themes of Part Two is the recognition of separable levels in solid modeling. The design of GWB is based on a low-level machinery for expressing computations on solid models in the form of Euler operators. On the top of this, higher layers are added, culminating in the advanced algorithms of Chapters 14 and 15.

The modeling concepts visible in each layer are different. In the implementation of Euler operators, considerable care must be put into managing all data items in the half-edge data structure properly. A user that expresses modeling operations in terms of low-level Euler operators must be concerned with providing correct parameters to the operators, and on navigating correctly in the data structure. Finally, solid primitives and set operations emphasize the correct decomposition of a complex part into simpler basic parts, and lower-level details are irrelevant. Clearly, each additional layer is based on additional higher-level, problem oriented modeling primitives.

The same applies for the final layer of the user interface. A successful user interface must be oriented towards the problems that are supposed to be dealt with by the system, and irrelevant details on how things actually take place on lower levels must be effectively hidden. In the analogy

to computer languages suggested in Chapter 8, the roles of the user interface and the so-called "very high-level languages" are similar: in both cases, the user must have natural tools for expressing his/her problem and what he/she wants to be performed, while the details of how this is to be accomplished should be of no concern to him/her.

The design of a good user interface is a demanding task that involves both "hard" technological aspects and "soft" human-oriented aspects. The list of topics that must be taken into account while developing a user interface includes the following:

1. *Conceptual model*: what are the objects the user can manipulate through the user interface?
2. *Functionality*: what operations the user can specify?
3. *Interaction style*: how can the user express his/her wishes by using certain (graphical) input devices?
4. *Context*: how are the inputs entered by the user mapped to the internal objects and operations of the interactive program?
5. *Feedback*: how are the effects of operations and the state of the interactive program represented for the user?
6. *Robustness*: how are errors detected, and recovered from?

A good user interface is based on a clear conceptual model of the application, and all its features are consistent with the model chosen. Like great food, a good user interface is more than just good ingredients put together. Good taste and sense of detail are as important for the developer of a user interface as they are for a chef.

In this text, we shall mainly concentrate on the design of the functionality of the user interface with less regard on the other items. In particular, we shall not deal with the problems of graphical interaction.

One of the assets of the layered modeling system architecture of GWB is that it is possible to develop user interfaces with widely varying modeling concepts. In this chapter, we shall exemplify this by developing two user interfaces to GWB, namely a batch interface based on CSG-style input, and an interactive interface based on "drawing" operations and sweeping.

17.2 BATCH INTERFACE

Stripped to its very essentials, a CSG interface involves three families of functions:


```
int  main(argc, argv)
int  argc;
char **argv;
{
    invert(block(atof(argv[2]), atof(argv[3]),
                 atof(argv[4])), argv[1]);
}
```

Program 17.1 The block primitive program.

1. functions to create primitive solids
2. functions to combine primitives by set operations
3. functions to translate and rotate primitives or combinations of primitives.

By virtue of the inversion algorithm, we can use files in secondary storage to represent the results of each of these groups of functions. In this design, primitive generators accept a few descriptive parameters, and create a file containing the desired object with the inversion algorithm. Set operations and other manipulations read their argument solids from files, and create new files as their results.

Based on the functionality developed in earlier chapters, the implementation of the programs is relatively straightforward. For instance, the program for creating a rectilinear block is implemented with the one-line procedure given in Program 17.1. According to UNIX conventions, the command line is available in the array *argv*; *argc* denotes the number of words in the command line. Here the dimensions of the block are retrieved from the second, third, and fourth argument with help of the UNIX standard procedure `atof` that converts a string of numerals to a floating-point number. The name of the part is taken from the first argument.

In this case, the user interface actually consists of the combination of the UNIX terminal command program, "shell," and programs such as 17.1. The user can consider each such program a "command" that obeys a certain syntax, defined by the way it treats the command line arguments. For instance, assuming that the compiled version of the Program 17.1 is stored in the executable file "`block`," its shell syntax is

```
block <file> <dx> <dy> <dz>
```

where `<file>` is the desired name the of the file for the part, and `<dx>` `<dy>` `<dz>` are the x-, y-, and z-dimensions of the block.

Shell syntax	*Description*
`block file dx dy dz [tx ty tz]`	Create block, store result in *file*
`cylx file r h [n] [tx ty tz]`	Create cylinder, axis = x
`cyly file r h [n] [tx ty tz]`	Create cylinder, axis = y
`cylz file r h [n] [tx ty tz]`	Create cylinder, axis = z
`conex file r h [n] [tx ty tz]`	Create cone, axis = x
`coney file r h [n] [tx ty tz]`	Create cone, axis = y
`conez file r h [n] [tx ty tz]`	Create cone, axis = z
`torusx file r1 r2 [n1 n2] [tx ty tz]`	Create torus, axis = x
`torusy file r1 r2 [n1 n2] [tx ty tz]`	Create torus, axis = y
`torusz file r1 r2 [n1 n2] [tx ty tz]`	Create torus, axis = z
`ball file r [n] [tx ty tz]`	Create sphere
`trans outfile infile tx ty tz`	Translate *infile* by tx, ty, tz
`rotat outfile infile rx ry rz`	Rotate *infile* by rx, ry, rz
`sect file x y z w`	Split *file* with plane $[x\ y\ z\ w]$
`slice outfile infile x y z w`	Calculate cross section
`union file1 file2 file3`	Calculate *file1* = *file2* $\cup$ *file3*
`inter file1 file2 file3`	Calculate *file1* = *file2* $\cap$ *file3*
`minus file1 file2 file3`	Calculate *file1* = *file2* $\setminus$ *file3*
`plot [-e ex ey ez] file1 file2 ...`	Draw *file1*, *file2*, ...
`plot3 file1 file2 ...`	Draw three views of *file1*, *file2*, ...
`volume file1 file2 ...`	Calculate volume of *file1*, *file2*, ...
`area file1 file2 ...`	Calculate area of *file1*, *file2*, ...

Table 17.1 Batch interface to GWB.

Table 17.1 lists a collection of programs that together implement a batch CSG interface to GWB. All these programs communicate through inversions. In addition to part names and size parameters, this specification includes optional parameters tx ty tz that define a translation for the part created. The optional parameter n indicates the number of faces used to represent a curved surface.

Observe that cylinders, cones, and tori can be generated in three different default orientations. Hence rigid transformations are needed only in "nonorthogonal" parts. The drawing program `plot` has optional parameters ex ey ez that determine the viewing direction. Program `sect` is assumed to store the results of the sectioning into files with some conventional names.

Note that many other operations necessary can be performed by ordinary UNIX tools, such as "`ls`" for listing parts, "`mv`" for renaming parts, and "`rm`" for removing parts.

Program 17.2 gives an example of the batch CSG interface. The "shell program" given creates a simple tray in terms of block and cylinder primitives. The operations in the left column create the "outside" of the tray by subtracting small blocks of material from a larger block, and adding cylinders so as to effect the rounding of its corners. Operations in the right column repeat the same sequence of operations for the "inside" of the tray. The final result is the set difference of the outside and the inside. Of course, by including a scaling program the inside could be created just by scaling the outside.

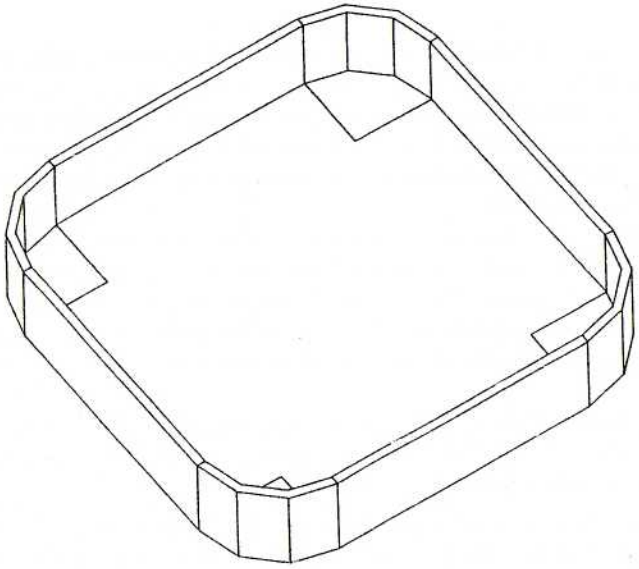
Observe that this description could be written to a file (say, "`tray`"), and evaluated by a single UNIX command

```
sh tray
```

in a batch processing manner. The file can be edited with conventional text editors to alter the design when necessary. Using the parameter substitution facilities of the UNIX shell the design could even be parameterized so as to model a family of trays of various basic dimensions.

17.3 INTERACTIVE INTERFACE

The programs of Table 17.1 can be used in a semi-interactive manner by typing commands and drawing intermediate results on a graphical display. However, it is inconvenient and inefficient to add other construction styles besides the CSG input in the batch approach. To include "drafting-type" input, a truly interactive program is needed.



```
block outside 100 100 20
block corn0 20 20 20
minus temp1 outside corn0
block corn1 20 20 20 80 0 0
minus temp2 temp1 corn1
block corn2 20 20 20 80 80 0
minus temp3 temp2 corn2
block corn3 20 20 20 0 80 0
minus temp4 temp3 corn3
cylz add0 20 20 20 20 0
union temp5 temp4 add0
cylz add1 20 20 80 20 0
union temp6 temp5 add1
cylz add2 20 20 80 80 0
union temp7 temp6 add2
cylz add3 20 20 20 80 0
union temp8 temp7 add3
```

```
block inside 96 96 18 2 2 2
block icorn0 20 20 20
minus itemp1 inside icorn0
block icorn1 20 20 20 80 0 0
minus itemp2 itemp1 icorn1
block icorn2 20 20 20 80 80 0
minus itemp3 itemp2 icorn2
block icorn3 20 20 20 0 80 0
minus itemp4 itemp3 icorn3
cylz iadd0 20 18 20 20 2
union itemp5 itemp4 iadd0
cylz iadd1 20 18 80 20 2
union itemp6 itemp5 iadd1
cylz iadd2 20 18 80 80 2
union itemp7 itemp6 iadd2
cylz iadd3 20 18 20 80 2
union itemp8 itemp7 iadd3
minus tray temp8 itemp8
```

Program 17.2 Example of the batch CSG interface.

17.3.1 Interaction Architecture

In analogy to programming languages and compilers, the input of any interactive program can be viewed as a computer language. Just like a programming language must be capable of describing objects (variables, data structures) and operations that can be performed on them (e.g., arithmetic expressions, assignments, and input/output), an interactive geometric program needs facilities for describing geometric objects and for specifying geometric operations on them.

This analogy can be exploited in the actual design of the interactive program. In particular, we can use many of the ideas and techniques developed for compiler construction [2] in the design and implementation of the program.

This approach to the construction of the interactive interface to GWB is summarized in Figure 17.1. The user types sentences of the command language according to the lexical conventions incorporated in a *lexical analyzer*. A *command language parser* recognizes these sentences, and invokes appropriate *command procedures* for performing the task described. All these components of the program use the data available in the half-edge data structure and certain state information to be described in the below.

By virtue of this general architecture, the design of the interactive program can be split into the following tasks:

1. Select a collection of *modeling primitives*, i.e., the basic objects that the user manipulates.

2. Based on the primitives chosen, develop a *command language* for creating primitives, and for performing manipulations on them. This task also includes the selection of the *state variables* and the design of adequate *echoing* of the effects of the commands.

3. Develop a *parser* for the command language, and *command procedures* that actually implement the operations specified.

4. Select the *interaction style*, i.e., the techniques the user can use to express sentences of the command language with the available (graphical) input devices such as a pointing device (mouse), function keys, and ordinary keyboard input. This task is comparable to developing a *lexical analyzer* for the command language parser.

We shall follow this approach, with emphasis on the first three items.

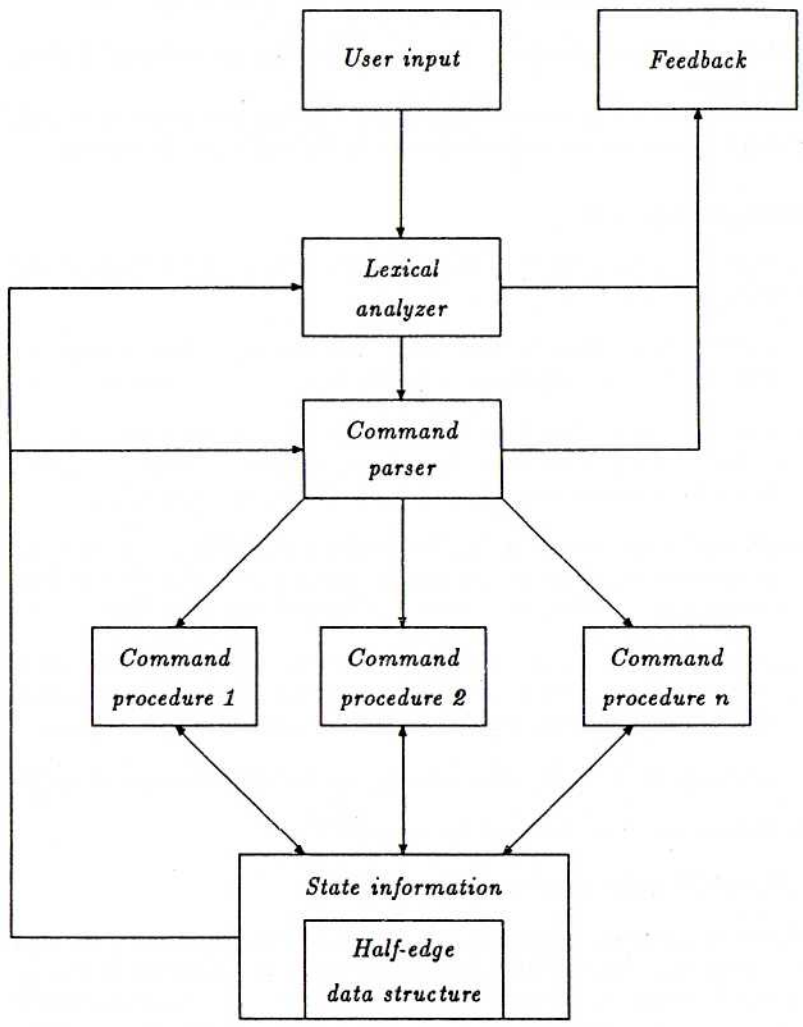


Figure 17.1 Architecture of the interactive interface.

17.3.2 Modeling Primitives

The interactive program is intended to give the user facilities for creating geometric objects by drawing outlines consisting of straight lines and circular arcs, by sweeping outlines, and by combining swept parts in terms of set operations. Clearly, in this case the modeling primitives of interest are

1. points,
2. lines,
3. arcs,
4. outlines, and
5. parts.

The primitives and their respective operations form a hierarchy: outlines consists of lines and arcs, and parts are created from outlines. Similarly, commands for creating and manipulating outlines will be concerned about individual modeling primitives (points, lines, and arcs), while sweeping operations deal with complete outlines, and set operations with complete parts.

17.3.3 Anatomy of a Command Language

A command generally involves a *verb*, the part that identifies the operation to be performed, *arguments* that identify the objects processed by the operations, and *parameters* that further describe the operation. For instance, the hypothetical command

```
LINE 100, 150/SOLID
```

consists of the verb LINE, the arguments 100 and 150, and the parameter SOLID. The remaining parts "<space>", ",", and "/" just separate the significant parts from each other and carry no other information.

State Information

In addition to explicit arguments and parameters, a command can use implicit state information. For instance, the starting point of the line created by the LINE command above could be implicitly defined to be the "current point." After the execution of the command, the new current point would be the point (100 150).

One of the most important issues in command language design is to decide what information will be included explicitly in a command, and

what information will be derived from the state information in effect at the time when the command is issued, or the *context* for short. A closely related problem to this is the *echoing*: if the interpretation of a command is dependent of the context, it must be indicated to the user. For instance, in the example above the current point could be indicated by a special highlighted cursor symbol.

The hierarchy of modeling primitives and their operations suggests a similar hierarchy in context information. Specifically, a context would consist of the following components:

1. There is always a "current view" (CV) that describes how the parts modeled are visible. The current view consists of a 4-by-4 transformation matrix from part coordinates to window coordinates, and the information defining a windowing operation from the window to the actual display.

2. There is always a "current object" (CO) being under construction. Moreover, there is a linear ordering of all parts created from the most recently created to the least recent part.

3. In the current part, there is always a "current face" (CF) on which the drawing operations take place. CF and all previous current faces have a linear ordering determined by the order they became current.

4. In the current face, there is always a "current point" (CP). Moreover, the points of the current face have a circular ordering determined by their connections to each other. (This ordering often coincides with the order the points were created.)

Part Commands

After deciding on the state variables, the operations of the command language can be specified by describing their effect on the state information. Hence, after recognizing that a pushdown stack is the natural data structure for recording parts, it is straightforward to specify operations related to parts as follows:

`open <name>` Assign name for a part. The next part created will be associated with the name given. If the user does not supply a part name for a part, a unique name is assigned to it by default.

`delete-part` Delete top of the part stack.

`union` Calculate the set union of the two topmost parts of the stack, push result onto the stack.

`minus` As `union`.

`inter` As `union`.

`select-part <name>` Make part become the top of the stack.

By virtue of the last operation, the user can modify the ordering of parts if he wants to.

Observe that in the above design, many commands consist of the verb part alone, because argument objects can be derived from the context.

Drawing Commands

The design of operations for points, lines, and arcs can also exploit the context in a similar fashion:

`move <u> <v>` If no point at $(u\ v)$ exists, create a new point at $(u\ v)$ and make it current. Otherwise, make the point at $(u\ v)$ current.

`line <u> <v>` Draw a line from the current point to a new point at $(u\ v)$, make the new point current. In addition, if the line divides the current face into two faces, select one of them as the new current face.

`arc-cw` Draw a circular arc in clockwise orientation. The arc is centered at the previous point of CP, and delimited by CP and the previous points of the center point. Select the current face as in `line`.

`arc-ccw` Draw a circular arc in counterclockwise orientation. The arc is centered at the previous point of CP, and delimited CP and the previous point of the center point. Select the current face as in `line`.

`next` Advance CP in the circular ordering, i.e. let CP = next point of CP.

`prev` Make new CP = the previous point of CP.

`del` Delete CP, make previous point current.

Observe that points have no names, but are identified indirectly through two coordinates u and v. The coordinates u and v are transformed to x, y, and z based on the current face and information of the current view (CV). Hence, the corresponding point x, y, and z is calculated by projecting the point u, v onto the current face along the viewing direction. This amounts to solving the unknown variables x, y, z from the three equations determined by the face equation of CF and the requirement that x, y, z transform to u, v under CV.

Outline Commands

The design is continued by sketching the operations for outlines. Again, a pushdown stack is the natural data structure for recording the state information.

sweep <depth> Sweep CF along its normal, make the "top" face of the swept region the new CF by pushing it onto the face stack.

select-face <u> <v> Make the nearest face that contains the point $(u\ v)$ current by pushing it onto the face stack.

prev-face Make previous current face current again by popping the stack.

hole As **prev-face**, but create first a hole by sweeping CF down through the object.

Note that commands for creating lines and arcs can also affect the current face information.

Viewing Commands

Finally, we need commands that select the current viewing operation:

eye <ex> <ey> <ez> Select the viewing direction.

window <ul> <vb> <ur> <vt> Select the window.

17.4 IMPLEMENTATION

In the UNIX programming environment the implementation of an interactive program can proceed nicely along the lines of Figure 17.1.

17.4.1 Lexical Analyzer

The commands outlined in the previous section consist of several types of components, including keywords, coordinates, and part identifiers. The task of the *lexical analyzer* is to recognize these *lexical tokens* from the input stream, and provide them to the parser. In our case, the lexical analyzer becomes a C function yylex that returns an integer code denoting the type of the lexical token read, and stores its actual value in a global variable yylval as described below. (The obscure names are due to conventions of the parser generator program we use for parser construction.)

Each keyword of the language is considered a distinct lexical token. For the purposes of lexical analysis, keywords and the literals denoting the


```
struct
{
      char        *word;
      int         value;
} w;

struct w keyw[] =
{
      "move",     MOVE,
      "line",     LINE,
      ...
      0,          0,
};
```

Program 17.3 Keywords of the command language.

token class codes can be represented in an array of records as shown in Program 17.3. Observe that is is easy to define abbreviations of keywords by associating several strings with the identical code.

The recognition of other tokens is based on *lexical conventions* that specify the formats of valid identifiers, numeric constants, and so on. For instance, identifiers are often required to have an alphabetic leading character (a ... z, A ... Z), and to consist entirely either of alphabetic characters or of numerals (0 ... 9). Under this rule, it is straightforward to distinguish identifiers from numeric constants that must start with a numeral or a unary minus.

Program 17.4 gives an implementation of the lexical analyzer that follows these rules. The analyzer reads user input in terms of the UNIX standard procedure **getchar** that returns the characters typed by the user one at a time. The while-loop ignores all newline characters ("\n"), spaces, and tabs ("\t"); hence these may be inserted in commands at will.

After reading the initial character of a token, the analyzer can immediately decide whether a character string (strget) or a numerical value (numget) should be read. Keywords are handled by searching a keyword array as defined in Program 17.3; if the token is found, the corresponding numeric value is returned to the parser. Otherwise, the string is assumed to be an identifier, and the code IDENTIFIER is returned. The actual string is sent forth by depositing a pointer to the string in the global variable *yylval* available to the parser. Observe that in this design, keywords cannot be used as identifiers.

Numeric values are handled by converting the string to a floating-point

```
extern int yylval;
int c = ' ';

yylex()
{
    int     i;
    int     ret;
    char    str[STRMAX];

    while(c == '\n' || c == ' ' || c == '\t') c = getchar();
    if(c >= 'a' && c <= 'z' || c >= 'A' && c <= 'Z')
    {
        strget(str);
        i = 0;
        while(keyw[i].value)
        {
            if(strcmp(keyw[i].word, str)) return(keyw[i].value);
            i++;
        }
        yylval = str;
        return(IDENTIFIER);
    }
    if(c == '-' || c >= '0' && c <= '9')
    {
        numget(str);
        yylval = atof(str);
        return(NUMBER);
    }
    ret = c;
    c = getchar();
    return(c);
}

strget(str)
char *str;
{
    ...
}

numget(str)
char *str;
{
    ...
}
```

Program 17.4 Lexical analyzer.

number with the standard UNIX procedure `atof`, and returning the literal NUMBER together with the actual value. All other characters are returned verbatim in terms of their ASCII code interpreted as a number 0...127.

17.4.2 Parser and Command Procedures

For the generation of the parser, the UNIX environment includes a very useful software tool YACC (Yet Another Compiler-Compiler) [61].

YACC—Yet Another Compiler-Compiler

YACC is a software tool for generating a parser from a specification of *grammar rules* and *actions* associated with the rules. The resulting parser may be included as a subroutine into the user's program. The user is also expected to supply a function `yylex` to act as a lexical analyzer.

The grammar rules are expressed in a style similar to the well-known Backus-Naur form (BNF). For instance, the syntax rule for command `window` is expressed as follows:

```
window:     WINDOW NUMBER NUMBER NUMBER NUMBER =
            {
                    window($2, $3, $4, $5);
                    newframe();
            }
```

In the above, ":" separates the name of the rule (i.e., a *nonterminal* symbol) from its body, and "=" separates the action from the body. The body consists of five terminal symbols ("WINDOW" and four occurrences of "NUMBER"). The body can also include names of other rules; in particular, it can include its own name to effect recursive syntax definitions. The action is just a C compound statement bounded by "{" and "}". In this case, the action sets the new window definition in effect and redraws the screen.

Terminal symbols are represented by literal values defined in terms of a `token` statement:

```
%token WINDOW NUMBER
```

By virtue of this definition, "WINDOW" and "NUMBER" become literals denoting some numeric values > 255. These literals can be used in the lexical analyzer in the matter shown in Programs 17.4 and 17.5.

The action part can access the values associated with the lexical tokens or rule names of its body through pseudovariables "$2", "$3", etc. that denote the value associated with the second, third, etc. token or name. The action part can associate a value to its own rule by assigning a value to the pseudovariable "$$".

YACC Example: Part Commands

Let us now examine the use of YACC with a more elaborate example. For this purpose, we select the part commands.

As noted in the above, a pushdown stack is the natural data structure for parts. Therefore, the first thing needed is the implementation of the stack and its basic routines. This is accomplished in Program 17.5.

"Parts" of this design are simply named solids. Procedure **pushpart** deposits a new part onto the stack; its name is copied with the UNIX library procedure **strcpy**. Procedure **poppart** removes a part from the stack, and returns a pointer to its solid node. Procedure **maketop** arranges a part to become the top of the stack by pushing it onto the stack, and removing the old entry. Procedure **strcmp** is again from the UNIX standard procedure library; it compares two character strings, and returns 1 if they are equal and 0 otherwise.

With these procedures, the part commands can be implemented by the YACC definitions of Program 17.6. In the program, "%{" and "%}" bound global definitions. Lexical tokens needed for these commands include the token "IDENTIFIER" and a token for each keyword; the actual keywords are specified as in Program 17.3.

By conventions of YACC, "%%" separates the foregoing definitions from the actual grammar rules. In this case, the first rule simply states that the language recognized by this parser consists of a list of **stat**'s. The rule for **stat** lists the statements of the language. In both rules, alternative bodies are separated by a "|".

The next six rules correspond with the six part commands. The action for rule **openpart** simply records the new part name; the required **pushpart(...)** is performed only after a solid actually is created. The action of the rule **deletepart** removes the topmost part, erases its image (**erasesolid**), and removes the solid (**rmsolid**). The code of **selectpart** makes a part to become the topmost one. Finally, set operations work by fetching the argument parts from the stack, and pushing the result back onto it. The display is updated by erasing the images of the argument solids and drawing the result with the procedure **displaysolid**. The name of the new part is fetched from *curname*; hence the user can give a name for the result in terms of **openpart** before invoking the set operation. Rules for **minus** and **inter** are analogous to **union**.

Main Program

From input such as that of Program 17.6, YACC creates a procedure called **yyparse**. To construct a working program, a main program that calls it


```
#define       MAXPNAME      30
#define       MAXPSTACK     30

typedef struct
{
      Solid    *psolid;
      char     pname[MAXPNAME];
}     Part;
Part  pstack[MAXPSTACK];
int   pstacktop = -1;

void  pushpart(s, name)
Solid *s;
char  *name;
{
      pstack[++pstacktop].psolid = s;
      strcpy(name, pstack[pstacktop].pname);
}

Solid *poppart()
{
      Solid          *top;
      top = pstack[pstacktop].psolid;
      pstacktop--;
      return(top);
}

void  maketop(name)
char  *name;
{
      for(i = 0; i < pstacktop; i++)
            if(strcmp(name, pstack[i].pname))
            {
                  pushpart(pstack[i].psolid, pstack[i].pname);
                  for(i++; i < pstacktop; i++)
                  {
                        pstack[i-1].psolid = pstack[i].psolid;
                        strcpy(pstack[i].pname, pstack[i-1].pname);
                  }
                  pstacktop--;
                  return;
            }
}
```

Program 17.5 Part stack operations.

```
%{
char        curname[30];
solid       *s, *sa, *sb, *res;
%}
%token      IDENTIFIER OPENPART DELETEPART
            SELECTPART UNION INTER MINUS
%%
stats:      stat | stats stat;
stat:       openpart | deletepart | selectpart |
            union | minus | inter;
openpart:   OPENPART IDENTIFIER =
            {
                 strcpy($2, curname);
            }
deletepart: DELETEPART =
            {
                 s = poppart();
                 erasesolid(s);
                 rmsolid(s);
            }
selectpart: SELECTPART IDENTIFIER =
            {
                 maketop($2);
            }
union:      UNION =
            {
                 sa = poppart();
                 sb = poppart();
                 erasesolid(sa);
                 erasesolid(sb);
                 res = setop(sa, sb, UNION);
                 displaysolid(res);
                 pushpart(res, curname);
            }
minus:      ...
inter:      ...
```

Program 17.6 Parser example.


```
int     main(argc, argv)
int     argc;
char    **argv;
{
    int         inputfd;

    if(argc > 1)
    {
        if((inputfd = open(argv[1], 0)) != -1)
        {
            close(0);
            dup(inputfd);
            close(inputfd);
        }
        else fprintf(stderr, "%s: cannot open %s\n",
                     argv[0], argv[1]);
    }
    yyparse();
}
```

Program 17.7 Main program.

must still be supplied. For the purposes of this text, the code given in Program 17.7 is sufficient.

The procedure makes it possible to use the interactive program also in a batch manner by allowing commands to be read from a file given as an optional argument in the shell command line of the program. This is accomplished by redirecting the terminal input to the file in terms of UNIX system calls `close` and `dup`.

17.5 NOTES

In practice, the two user interfaces outlined in this chapter can be tied together by giving the interactive program a facility to write and read inversions, and including the CSG functions into it.

The command language was developed so as to facilitate the use of graphical interaction, when available. Many commands do not take any arguments at all, and can be implemented by a menu selection or function keys. Some others require only two-dimensional position information, and can be implemented by a combination of pointing and menu selection. Hence, the only commands that require typing are the ones involving part

names; they are expected to occur relatively infrequently. Consult any good textbook in computer graphics [88,39] for interactive graphics techniques. See also [77] for one particular approach to the construction of an interactive modeler based on a GWB-like system.

Good tools are indispensable when implementing a nontrivial program. According to the author's experience on the use of parser generators, they can greatly aid the development of virtually any program that has to deal with a collection of commands. When tools such as YACC are available, their use is strongly recommended.

PROBLEMS

17.1. The user interface described in the text is based on having a collection of "current" entities which are worked on. A more versatile approach to constructing the state information of a "drawing-oriented" user interface is to use a separate *selection buffer* for managing a collection of "selected" graphical entities. The user can edit the buffer, i.e., add or remove items from the buffer, and apply operations to entities stored in the buffer.

Outline an implementation of a selection buffer that can hold any number of half-edge data structure instances. Specify the procedures needed for editing the contents of the buffer.

17.2. Based on the selection buffer of Problem 17.1, outline a command language that supports the similar operations as those described in Section 17.3, but are based on the buffer rather than the "rigid" state information outlined in the text.

17.3. Half-edge data structures are versatile enough to support also drafting-type operations of the kind discussed in Chapter 2. Outline a command language that could be used to draw simple two-dimensional figures consisting of points, lines, and arcs.

Hint: Single points could be modeled as a half-edge data structure consisting of a single face and vertex. Single lines could be modeled as one-edge solids. Observe that many of the procedures of Chapter 13 are applicable also for such objects.